

MOATLY

Moat App — Technical Breakdown

A mobile-side implementation plan for sections 5.1, 5.2, 5.3, 5.6, 5.9 of the Build Brief v1.0

Prepared for: Engineering lead, Moatly
Document version: 0.1 · Status: Draft for review
Date: April 2026

0. How to read this document

This is a **technical breakdown** of the Moat mobile application — the piece of the Moatly system described in sections 5.1, 5.2, 5.3, 5.6, and 5.9 of the Build Brief. It does not cover the backend Go service, the nRF52840 firmware, or the Unity ritual app. Those have their own breakdowns.

The audience is the team that will actually write the code. This document assumes the Build Brief has been read and its non-negotiable principles (section 2 of the brief) are accepted as constraints, not preferences.

The document is ordered so that a reader can stop at the end of any section and still have a usable picture. Sections 1–3 are for everyone (decisions, architecture, module map). Sections 4–7 are the detailed per-module specs for Phase 1 — the MVP. Sections 8–10 cover cross-cutting concerns, implementation order, and what is deliberately out of scope for this first breakdown.

0.1 What this document contains

- Decisions that must be made before Week 1 of implementation.
- A layered app architecture with a concrete module map and directory layout.
- Per-module specifications for the Phase 1 (app-only MVP) modules.
- Cross-cutting specs for key lifecycle, state management, and offline behaviour.
- An implementation order mapped to the Build Brief's Phase 1 exit gate.

0.2 What this document does not yet cover

- Detailed module specs for Phase 2 (Moat service client, Megolm via vodozemac FFI, Major Statement UI, complaint flow, WebSocket stream) — produced as a separate breakdown when Phase 1 exits.
- Detailed module specs for Phase 3 (BLE/device integration, combination UI, composite identity code, glow driver, virtual device) — same.
- Backend API implementation. Covered by a backend-side breakdown.
- Firmware. Covered by a firmware-side breakdown.

0.3 Document map

§	Contents	Primary reader
1	Decisions required before Week 1.	Lead, founder
2	App architecture, module map, directory layout.	All engineers
3	The Phase 1 module catalogue — what ships in the MVP.	Lead, mobile leads
4	Per-module specifications (Phase 1 modules).	Mobile engineers
5	Cross-cutting: key lifecycle, state, navigation, offline, errors.	Mobile engineers

§	Contents	Primary reader
6	Data flows for the user-visible scenarios.	Mobile + QA
7	Test strategy for the mobile app.	QA, lead
8	Implementation order — mapped to Build Brief Phase 1 exit gate.	Lead, PM
9	Open questions for decision this week.	Lead, founder
10	What comes next (Phase 2 and Phase 3 follow-up breakdowns).	Lead

1. Decisions required before Week 1

Seven decisions listed below. Each blocks a non-trivial chunk of Phase 1 work. They are here because the Build Brief marked them 'decision required' or because they have surfaced during this breakdown.

For each decision the default recommendation is stated first, followed by the trade-off summary. The engineering lead should either ratify the default or overrule it in writing before scaffolding starts.

1.1 Framework — React Native (Expo) vs Flutter

Recommendation: React Native with the Expo prebuild workflow.

The decisive factor is the vodozemac FFI. vodozemac is a Rust library and the path to consuming it in mobile is through a generated FFI (UniFFI, cxx-rs, or a hand-written binding). UniFFI has mature React Native support via uniffi-bindgen-react-native; Flutter's Rust FFI path (flutter_rust_bridge) works but is a thinner ecosystem. Both are buildable; RN has more exemplars in the Matrix / Element codebases we will cross-reference.

Secondary factors: TypeScript is the dominant mobile skill in the current hiring market; the React Native BLE library (react-native-ble-plx) is comprehensive and is already used by Nordic in reference apps; Reanimated 3 handles the ritual-adjacent animation budget comfortably.

*Choosing Flutter instead is defensible if and only if the lead has strong prior Flutter experience and a Rust-FFI exemplar they have shipped. The brief's **recommendation** was RN-or-Flutter with team skill as the tie-breaker; this breakdown picks RN for the FFI reason above.*

1.2 Expo prebuild vs bare React Native

Recommendation: Expo prebuild (the 'managed-config, bare-build' model).

We need native modules for BLE, SQLCipher, hardware keystore, and the vodozemac Rust crate. That rules out Expo Go (managed workflow). Expo prebuild keeps the Expo config system and developer ergonomics while letting us land arbitrary native code, which is exactly our situation. Going fully bare (React Native CLI only) is possible but we lose Expo's config plugins, EAS build cache, and the well-maintained expo-sqlite/expo-crypto/expo-secure-store libraries that we will use anyway.

1.3 State management

Recommendation: Zustand for global UI state, React Query for server state, plain React for local component state.

The brief lists Zustand. Adding React Query is cheap and separates 'data that lives on the server' from 'data that lives in the app.' This matters here because the server state has novel retention semantics (seven-day deletion, session rotation at cycle boundary) and benefits from a caching layer with explicit invalidation. Redux is avoided — the boilerplate is not justified at our scale.

1.4 Navigation

Recommendation: React Navigation (native stack + bottom tabs). No deep-linking to Moat content.

Standard choice. We explicitly disable deep linking into Moat screens — opening a notification must not deep-link to any content view. Opening the app always lands in the same known state, based on the user's progression (onboarded-or-not, paired-or-not, in-a-Moat-or-not).

1.5 Local database

Recommendation: op-sqlite with the SQLCipher build. SQL written by hand. No ORM.

expo-sqlite does not bundle SQLCipher. op-sqlite (@op-engineering/op-sqlite) does, is well maintained, supports the New Architecture, and is faster than the alternatives. No ORM: the SQL is small, we want to read it line by line during security review, and our schema has no joins that would benefit from a query builder.

1.6 Email validation and linkage-token storage

Recommendation: Email is optional at onboarding; prompted for separately after covenant acceptance; stored in expo-secure-store keyed by account ID, not in the main SQLCipher DB.

The brief treats email as a separate concern (section 5.7) and the server stores it in a separate table that can be deleted independently. Mirroring that separation on-device makes the 'delete my email address' user action trivial and avoids accidentally coupling email deletion to the main DB's encryption key lifecycle.

1.7 Testing toolchain

Recommendation: Jest for unit; Maestro for E2E; Detox as a fallback if Maestro hits limits; fast-check for crypto property tests.

Maestro is declarative, fast, runs both iOS and Android from one YAML, and integrates with CI cheaply. Detox is more capable for gnarly scenarios (BLE, hardware keystore prompts) and is the fallback for Phase 3.

1.8 Summary table

Decision	Default recommendation	Owner
Framework	React Native (Expo prebuild)	Lead
Workflow	Expo prebuild, not Expo Go	Lead
State management	Zustand + React Query	Lead
Navigation	React Navigation, no deep links	Lead
Local DB	op-sqlite with SQLCipher	Lead
Email storage	Secure store, separate from main DB	Lead
Testing	Jest + Maestro, fast-check for crypto	QA + lead

2. App architecture

The Moat app is built as four concentric layers. Code in an outer layer depends on code in an inner layer but never the reverse. Crypto and key material live in the innermost layer and are reached only through narrow, well-typed interfaces.

2.1 Layered architecture

Layer 4 – UI

Screens, components, navigation, animation. Uses hooks from L3.
No direct DB, crypto, BLE, or network calls.

Layer 3 – Domain

Use cases: "acceptCovenant", "postMajorStatement", "rotateSeed".
Stores (Zustand), React Query mutations / queries, business rules.
Pure TypeScript; no React; fully unit-testable.

Layer 2 – Infrastructure

API client, WebSocket client, BLE adapter, SQLCipher DAO,
push-notification adapter, Postmark relay client.
Transforms wire-format ↔ domain types. No React.

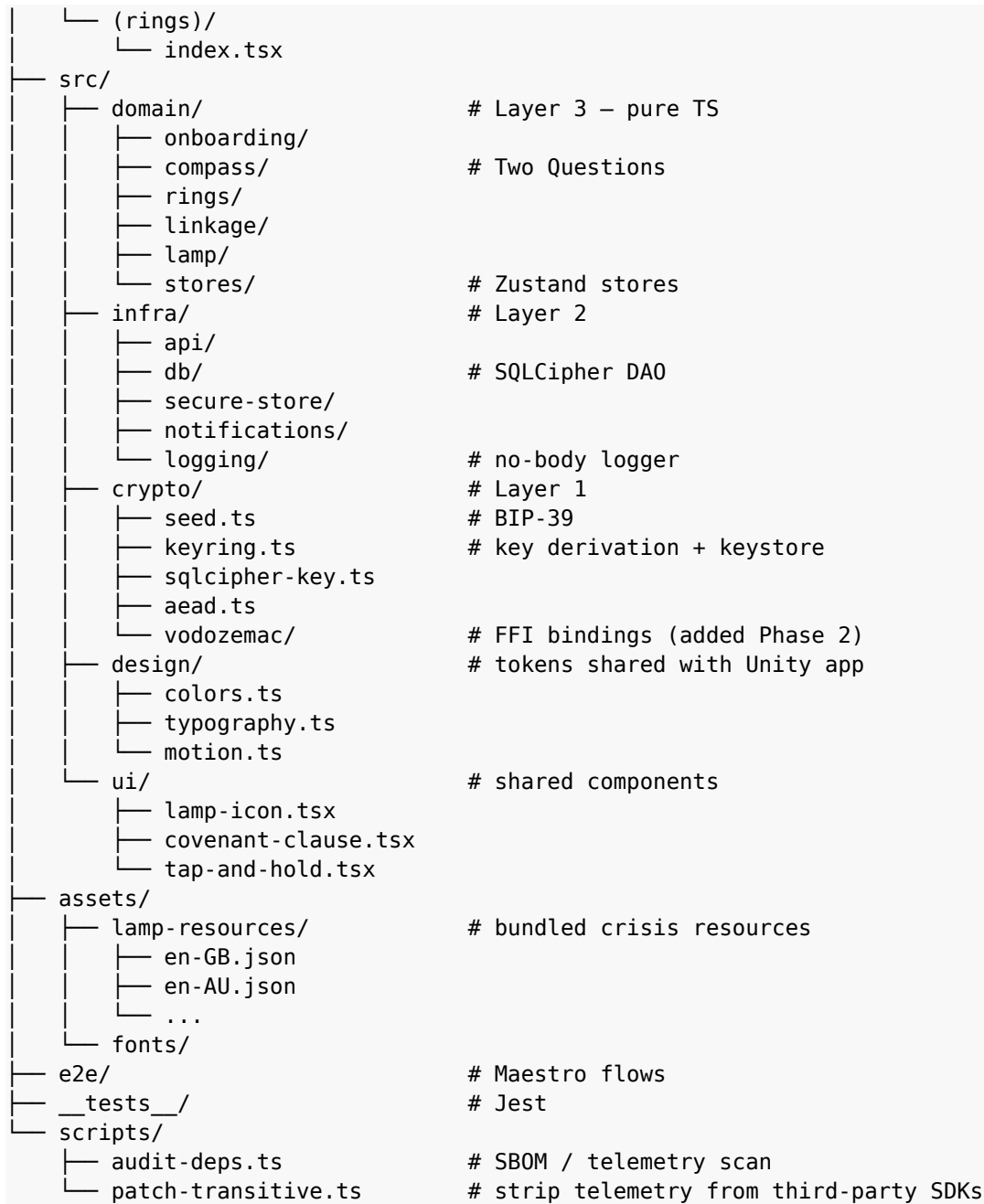
Layer 1 – Crypto core

vodozemac FFI boundary, BIP-39 seed derivation,
SQLCipher key derivation, AEAD helpers, keystore interface.
Only layer that touches raw key material. Minimal surface.
Wrapped by test doubles for upper-layer tests.

*A rule that is worth restating: **no layer may skip another**. The UI never imports anything from Layers 1 or 2. The Domain never imports anything from Layer 1. This keeps the crypto surface small, which matters for external audit.*

2.2 Directory layout

```
moat-app/
├── app/                                # Expo Router screens (Layer 4)
│   ├── _layout.tsx
│   ├── (onboarding)/
│   │   ├── threshold.tsx
│   │   ├── declaration.tsx
│   │   ├── covenant.tsx
│   │   ├── device-pairing.tsx
│   │   ├── seed-phrase.tsx
│   │   ├── two-questions.tsx
│   │   └── placement.tsx
│   ├── (moat)/
│   │   └── index.tsx                # stub in Phase 1
│   └── (lamp)/
│       └── index.tsx
```



2.3 Module catalogue — Phase 1 scope

Modules are listed in rough dependency order (downstream modules depend on modules above them).

Module	Layer	Responsibility	Section
crypto.seed	1	BIP-39 seed generation, validation, word prompts.	§4.1
crypto.keyring	1	Seed → master key → derived keys (DB, identity). Keystore.	§4.2

Module	Layer	Responsibility	Section
infra.db	2	SQLCipher DB, DAO, migrations, key hand-off.	§4.3
infra.secure-store	2	OS keychain/keystore wrapper for the master key and tokens.	§4.4
domain.onboarding	3	The onboarding state machine; sequences steps 1–7.	§4.5
ui.covenant	4	Declaration + Covenant screens; tap-and-hold component.	§4.6
domain.compass	3	Two Questions: storage, 24h lock, cycle linking.	§4.7
domain.rings	3	Local ring ledger; ring computation (no server).	§4.8
domain.linkage	3	Unity handoff token; email-relay client; receipts.	§4.9
ui.lamp	4	The persistent safety surface. Offline, untracked.	§4.10
infra.logging	2	Body-stripping logger; boundary enforcement.	§4.11
infra.notifications	2	Silent push registration, no body payload ever.	§4.12

2.4 What Phase 1 intentionally leaves stubbed

- No Moat screen beyond a placeholder. No messaging anywhere.
- No vodozemac integration. The FFI scaffold is started in Phase 2.
- No BLE code. The device module is Phase 3.
- No WebSocket client. Phase 2.
- No complaint flow. Phase 2.
- No device pairing screen — the onboarding flow skips from 'declaration' straight to 'seed phrase' in Phase 1, and falls through to a 'virtual device' placeholder. The screen exists but does nothing cryptographic yet.

3. Phase 1 module catalogue

The remainder of this document specifies the modules listed in §2.3, in enough detail that each can be picked up by a single engineer and implemented against an acceptance spec. The specs are deliberately prescriptive about interfaces; they are deliberately non-prescriptive about internal implementation.

Every module spec contains the same sections: Responsibility, Public interface, Data, Dependencies, Notes, Risks, Acceptance criteria. Where a section is empty it is noted explicitly rather than omitted, so the reader can tell 'nothing needed here' from 'I forgot to write this'.

4. Per-module specifications

4.1 crypto.seed — BIP-39 seed phrase

Responsibility

Generate a 24-word BIP-39 English seed phrase from a cryptographically secure RNG on the device. Validate a 24-word phrase on re-entry. Emit three random word prompts for confirmation during onboarding. Never transmit the seed phrase or any plaintext derivative to the server.

Public interface

```
export function generateSeedPhrase(): string[];           // 24 words
export function validateSeedPhrase(words: string[]): boolean;
export function randomWordPrompts(
  seed: string[],
  count: number = 3
): { position: number }[]; // e.g. [{position: 7}, {position: 14}, {position: 22}]
export function verifyWordAt(
  seed: string[],
  position: number,
  provided: string
): boolean;
```

Data

None persisted by this module. The seed phrase leaves this module only as the argument to `crypto.keyring.deriveMasterKey` (§4.2).

Dependencies

- `react-native-get-random-values` for CSPRNG backed by the OS.
- `@scure/bip39` for wordlist and mnemonic validation.

Notes

- Use BIP-39 English wordlist only. We do not localise the seed phrase; localising reduces cross-device portability and adds no user benefit in this domain.
- Word prompts are rendered with a dropdown of 6 candidates (the correct word plus 5 random words from the wordlist) rather than a free-text input. Free text invites typos and leaks word shape to anyone looking over the user's shoulder.

Risks

- The OS CSPRNG on Android emulators is sometimes misbehaving. Always run the generator on real devices for the seed-phrase quality test.

Acceptance criteria

- `generateSeedPhrase` produces 24 distinct words from the BIP-39 list, with a 264-bit entropy check in a property test (fast-check, 10,000 iterations).
- `validateSeedPhrase` returns true for a fresh `generateSeedPhrase` output and false for any single-word substitution.

- Word prompts never ask for the same position twice; the three positions are chosen without replacement.

4.2 crypto.keyring — key derivation and storage

Responsibility

Derive the user's master key from the 24-word seed phrase. From the master key derive two subsidiary keys: the SQLCipher database key (used by infra.db) and the identity key (used later for Olm identity in Phase 2). Store the master key in the OS keychain/keystore with biometric protection. Never write the seed phrase itself to any persistent store.

Public interface

```
export type MasterKeyHandle = { __brand: 'MasterKeyHandle' };

export async function deriveAndStoreMasterKey(
  seed: string[]
): Promise<MasterKeyHandle>;

export async function loadMasterKey(): Promise<MasterKeyHandle | null>;

export async function deriveSubKey(
  handle: MasterKeyHandle,
  purpose: 'sqlcipher' | 'identity' | 'linkage-token'
): Promise<Uint8Array>; // 32 bytes

export async function wipeMasterKey(): Promise<void>;

export async function verifyAgainstSeed(
  handle: MasterKeyHandle,
  seed: string[]
): Promise<boolean>;
```

Data

Master key is stored via expo-secure-store with `requireAuthentication: true` on both platforms. The key is 32 bytes, derived from the seed phrase via PBKDF2-SHA512 (600,000 iterations) and then HKDF-expanded to per-purpose keys.

The `MasterKeyHandle` type is a *branded* empty object. The actual key bytes never leave this module; `subKey` derivation happens inside the module and returns the derived bytes only. This keeps raw key material out of garbage-collected JavaScript strings.

Dependencies

- expo-secure-store
- @noble/hashes for PBKDF2 and HKDF (audited, pure JS, deliberately not native for reviewability)

Notes

- HKDF salt is a constant per purpose ('moatly.sqlcipher.v1' / 'moatly.identity.v1' / 'moatly.linkage.v1'). Constants are versioned so that a future key-derivation change can coexist with old keys during migration.

- Biometric failure must not lock the user out permanently. The fallback is to require the seed phrase from the journal.

Risks

- Mobile keystore behaviour differs between iOS (Secure Enclave) and Android (hardware-backed Keystore, but device-dependent). Phase 1 test matrix must include at least one device per category (iPhone SE 3rd gen, Pixel 7, Samsung Galaxy mid-range, Samsung low-end with software-only keystore).

Acceptance criteria

- `deriveAndStoreMasterKey` with the same seed produces the same key on every invocation (verified via a test vector).
- `deriveSubKey` is deterministic for the same (handle, purpose).
- `loadMasterKey` after app restart returns the same handle (same underlying bytes) without requiring the seed phrase.
- `wipeMasterKey` followed by `loadMasterKey` returns null.
- No raw key material is ever written to `AsyncStorage`, the `SQLCipher` DB, the filesystem, or a log.

4.3 infra.db — SQLCipher database

Responsibility

Own the on-device encrypted database. Initialise `SQLCipher` with the key from `crypto.keyring`, run migrations, expose DAOs for each domain module, and provide a single commit/rollback boundary for multi-statement operations.

Public interface

```
export async function openDatabase(
  masterKey: MasterKeyHandle
): Promise<Database>;

export interface Database {
  close(): Promise<void>;
  transaction<T>(fn: (tx: Tx) => Promise<T>): Promise<T>;
  compass: CompassDao;
  rings: RingsDao;
  linkage: LinkageDao;
  onboarding: OnboardingDao;
}
```

Data

The Phase 1 schema (full SQL in `src/infra/db/migrations/001-init.sql`):

```
CREATE TABLE onboarding_state (
  id
    INTEGER PRIMARY KEY CHECK (id = 1),
  threshold_seen_at
    INTEGER,
  declaration_read_at
    INTEGER,
  covenant_accepted_at
    INTEGER,
  covenant_hash
    BLOB,
  seed_confirmed_at
    INTEGER,
```

```

    two_questions_completed_at INTEGER,
    placement_confirmed_at    INTEGER
);

CREATE TABLE user_compass (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    cycle_number  INTEGER NOT NULL,
    what_to_fix   TEXT NOT NULL CHECK (length(what_to_fix) <= 240),
    why_for_whom  TEXT NOT NULL CHECK (length(why_for_whom) <= 240),
    created_at    INTEGER NOT NULL,
    locked_until  INTEGER NOT NULL
);

CREATE TABLE ring_ledger (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    cycle_number  INTEGER NOT NULL,
    event_type     TEXT NOT NULL,          -- 'major_statement', 'witness',
                                           -- 'ritual_day', 'breakthrough',
                                           -- 'hard_season_stayed'

    event_at      INTEGER NOT NULL,
    payload       BLOB                    -- optional per-event data
);
CREATE INDEX idx_ring_ledger_cycle ON ring_ledger (cycle_number);

CREATE TABLE linkage_receipts (
    relay_id      TEXT PRIMARY KEY,
    account_id    TEXT NOT NULL,
    accepted_at   INTEGER NOT NULL
);

-- Intentionally no table supports ranking or cross-user queries.
-- This is schema-level enforcement of non-negotiable principle 7.

```

Dependencies

- @op-engineering/op-sqlite (SQLCipher build)
- crypto.keyring (§4.2) for the DB key

Notes

- Migrations are forward-only. There is no schema rollback. A breaking migration ships a compensating data-copy step.
- The entire DB is encrypted, so 'encrypt sensitive columns' has no meaning here — there are no less-sensitive columns. Everything inside is already confidential.
- Integrity-check the DB on open (PRAGMA integrity_check). A failed check means corrupt DB; treat as fatal, force re-onboarding with seed phrase.

Risks

- op-sqlite major version bumps sometimes break the native build. Pin to a minor version and audit upgrades.

Acceptance criteria

- Database file on disk is unreadable without the key (verified by an E2E test that takes a copy of the file, attempts to open with vanilla sqlite, and expects failure).
- Transactions roll back on any exception thrown inside the transaction function.
- A CREATE TABLE statement without a reviewed migration file fails CI.

4.4 infra.secure-store — OS keychain / keystore

Responsibility

Narrow wrapper around expo-secure-store, providing typed keys and enforcing that biometric-protected items are always accessed via an explicit unlock call rather than opportunistically. This module is the only caller of expo-secure-store in the entire app.

Public interface

```
type SecureKey =
  | 'moatly.master-key'
  | 'moatly.email-address'
  | 'moatly.unity-linkage-token'
  | 'moatly.account-id';

export async function set(key: SecureKey, value: Uint8Array): Promise<void>;
export async function get(key: SecureKey): Promise<Uint8Array | null>;
export async function remove(key: SecureKey): Promise<void>;
export async function clearAll(): Promise<void>; // account deletion
```

Data

No in-process cache. Every read hits the keystore. This is intentional — caching master-key bytes in a JavaScript variable is the mistake this wrapper exists to prevent.

Dependencies

- expo-secure-store

Notes

- ESLint rule (bespoke) forbids any `import ... from 'expo-secure-store'` outside this module.

Risks

- Biometric prompts are modal UX. Any call to `get()` that triggers a prompt during app startup will deadlock if not handled correctly. Use a single 'unlock' call at startup, not per-read prompts.

Acceptance criteria

- Every production call path is traced to verify no value is cached after retrieval.
- `clearAll` leaves zero items under the moatly prefix (verified on both platforms).

4.5 domain.onboarding — the onboarding state machine

Responsibility

Model the seven-step onboarding flow as an explicit state machine. Advance only on completion of each step. Persist each step completion to `onboarding_state`. Make it impossible — not just UI-difficult — to skip a step.

Public interface

```
export type OnboardingStep =
  | 'threshold'
  | 'declaration'
  | 'covenant'
  | 'device_pairing'
  | 'seed_phrase'
  | 'two_questions'
  | 'placement';

export type OnboardingState = {
  current: OnboardingStep;
  completed: OnboardingStep[];
  covenantHash: Uint8Array | null;
};

export function useOnboardingState(): OnboardingState;
export async function markStepComplete(step: OnboardingStep): Promise<void>;
export async function resetOnboarding(): Promise<void>;
export function isOnboarded(state: OnboardingState): boolean;
```

Data

Single-row `onboarding_state` table (§4.3). Timestamps for each step; `covenant_hash` for the covenant text that was accepted.

Dependencies

- `infra.db` (§4.3)

Notes

- Step transitions are validated server-side-free (there is no server in Phase 1). The transition rules live in a pure TS function that is 100% unit-tested.
- `markStepComplete` is idempotent — calling it twice with the same step is a no-op.
- `resetOnboarding` is exposed for 'start over from seed phrase recovery' and for developer builds. It wipes `onboarding_state` but not the master key — that's a separate action.

Risks

- Users who background the app mid-onboarding must resume exactly where they left off. Persist the current step before animating transitions, not after.

Acceptance criteria

- A Maestro E2E test attempts to deep-link to each screen and confirms that only the step matching the persisted current state renders; all others redirect back.

- `isOnboarded` returns true iff all seven step timestamps are non-null AND the covenant hash matches the current covenant text.

4.6 `ui.covenant` — declaration, covenant, and the tap-and-hold component

Responsibility

Two screens and one bespoke component. The Declaration screen is a scrollable reader for the seven rights — scroll-to-end is required before the continue button enables. The Covenant screen presents three clauses, each accepted via a deliberate 1.5-second tap-and-hold.

Public interface

```
// Screens are Expo Router entries; no programmatic API.
// The reusable component exported from src/ui/tap-and-hold.tsx:

export interface TapAndHoldProps {
  label: string;
  durationMs?: number; // default 1500
  onComplete: () => void;
  accessibilityHint: string;
}
export const TapAndHold: React.FC<TapAndHoldProps>;
```

Data

The three covenant clauses are stored in `src/domain/onboarding/covenant-text-v1.ts` as a string constant. Its SHA-256 is the `covenant_hash` recorded at acceptance. The text is loaded from code, not from a bundled file or the network, so that any change ships as a code review.

Dependencies

- `react-native-reanimated` 3 for the hold-animation
- `domain.onboarding` (§4.5) for `markStepComplete`

Notes

- **Accessibility:** the hold duration can be bypassed via the OS 'accessibility shortcuts' settings (VoiceOver / TalkBack). When a screen reader is active, the interaction is replaced by a standard accept button with an explicit spoken description of what acceptance means. This is the correct trade-off: the ritual friction is for able-bodied users, not a gate.
- No haptic on release. Haptic on start (subtle tick) and on completion (single firm thump). Three clauses, three thumps, done.

Risks

- Tap-and-hold on iOS fights with system gesture recognisers (back-swipe, control centre). Test on both notched and non-notched iPhones.

Acceptance criteria

- The continue button on Declaration does not enable until the user has scrolled to the last rendered line.

- A tap that lifts before 1.5s does not register the clause.
- All three clauses must be accepted for the Covenant step to complete; there is no UI state in which fewer than three are accepted but the user advances.
- The recorded covenant_hash matches the SHA-256 of the exact string constant version.

4.7 domain.compass — the Two Questions

Responsibility

Store the user's answers to 'What do you want to fix?' and 'Why? For whom? To what end?'. Enforce the 24-hour edit lock after submission. Associate each answer with a cycle number (cycle 0 during Phase 1 when no Moats exist yet).

Public interface

```
export interface Compass {
  cycleNumber: number;
  whatToFix: string;
  whyForWhom: string;
  createdAt: number;
  lockedUntil: number;
}

export async function submitCompass(
  whatToFix: string,
  whyForWhom: string,
  cycleNumber: number
): Promise<Compass>;

export async function loadCurrentCompass(
  cycleNumber: number
): Promise<Compass | null>;

export async function canEdit(compass: Compass): Promise<boolean>;

export async function amendCompass(
  compass: Compass,
  whatToFix: string,
  whyForWhom: string
): Promise<Compass>; // throws if !canEdit
```

Data

user_compass table (§4.3). Each cycle's compass is one row. locked_until is createdAt + 24h.

Dependencies

- infra.db (§4.3)

Notes

- Character limit (240 per field) is enforced on both the text input component and the DAO (CHECK constraint in the schema). Two enforcement points because one will eventually be wrong.

- The 24-hour lock is not a UI disable; it is a thrown error in `amendCompass`. The UI checks `canEdit` and greys out the edit affordance accordingly.

Risks

- Clock skew — `lockedUntil` uses device time. A user who changes their device clock can bypass the lock. This is considered acceptable (self-harm only) and is not mitigated.

Acceptance criteria

- `submitCompass` rejects any string over 240 UTF-16 code units before hitting the DB.
- `amendCompass` within 24h of creation throws.
- `amendCompass` after 24h succeeds and updates `createdAt` + `lockedUntil` atomically.

4.8 domain.rings — the local ring ledger

Responsibility

In Phase 1 the rings have no Moat events to feed them; they exist only for the ritual-day arc (blue) and the fire-scar arc (hard-season-stayed). The module ships the full computation function so Phase 2 does not need to change `domain.rings` — only its inputs.

Public interface

```
export type ArcColour = 'gold' | 'green' | 'blue' | 'grey' | 'fire';

export interface RingEvent {
  cycleNumber: number;
  eventType: 'major_statement' | 'witness' | 'ritual_day' |
    'breakthrough' | 'hard_season_stayed';
  eventAt: number;
  payload?: Uint8Array;
}

export async function recordEvent(event: RingEvent): Promise<void>;
export async function loadEventsForCycle(
  cycleNumber: number
): Promise<RingEvent[]>;
export function computeRingArcs(events: RingEvent[]): ArcColour[];
```

Data

`ring_ledger` table (§4.3). One row per event. `computeRingArcs` is pure and takes the rows as input.

Dependencies

- `infra.db` (§4.3)

Notes

- `computeRingArcs` encodes the arc rules from brief §5.6.1 as a single pure function. It is exhaustively property-tested.
- *The schema has no foreign key to any other user. There is no query like 'rings where user = ...'. This is the 'schema absence' form of principle 7 described in the brief.*

Risks

- Cycle number must align with the Moat service's cycle number once Phase 2 arrives. Until then, Phase 1 cycle numbers are locally assigned starting from 0 and will need a one-time migration in Phase 2.

Acceptance criteria

- No API endpoint in Phase 1 reads ring events from the server, and none will in any later phase.
- `computeRingArcs` is deterministic for the same event set.
- A property test confirms that no input event combination yields more than the per-cycle caps (1 gold, 1 green, 1 blue, 1 fire, unavoidable grey).

4.9 domain.linkage — Unity ritual app handoff and email relay

Responsibility

Mint an opaque account-ID token at first launch of the Unity app. Accept prescription envelopes from the Unity app and forward them via the backend's email relay (POST `/v1/linkage/relay_prescription` per §5.3.3 of the brief). Store receipt metadata locally so the user can see 'your last 7 prescriptions were delivered.'

Public interface

```
export async function issueUnityLinkageToken(): Promise<string>; // opaque

export async function relayPrescription(envelope: {
  accountId: string;
  playthroughRef: string;
  subject: string;
  bodyPlain: string;
  bodyHtml?: string;
}): Promise<{ relayId: string; acceptedAt: string }>;

export async function listRecentReceipts(
  limit?: number
): Promise<{ relayId: string; acceptedAt: number }[]>;
```

Data

`linkage_receipts` table (§4.3). The body is never written to the DB; only `relay_id`, `account_id`, `accepted_at`.

Dependencies

- `infra.db` (§4.3)
- `infra.api` (backend POST `/v1/linkage/relay_prescription`)
- `infra.secure-store` (§4.4) for the Unity linkage token

Notes

- The `playthroughRef` from the Unity app is not interpreted here. It is passed through and discarded — we never learn what was in the ritual.

- Rate limit (1/24h per account) is enforced server-side. The client displays a 'one more prescription allowed in 4h 12m' affordance based on the most recent receipt — to avoid a user triggering a 429.

Risks

- The Unity app is a separate build managed by a separate team. Schema drift on the envelope is a real risk. Add a schema-version check on both sides and fail closed (drop the envelope and log a single metric) rather than silently forwarding broken content.

Acceptance criteria

- relayPrescription never writes bodyPlain or bodyHtml to disk (verified by an E2E test that inspects the DB file and the filesystem for a known magic string after a relay).
- The Unity linkage token is 128 random bits, base64url-encoded, stored only in secure store.
- listRecentReceipts returns rows ordered by acceptedAt desc.

4.10 ui.lamp — the persistent safety surface

Responsibility

The Lamp icon in the corner of every Moat-related screen. Tapping it opens a full-screen, offline, locale-aware list of verified crisis resources. The Lamp has zero observability — no telemetry, no analytics, no log line, no server call, ever.

Public interface

```
// Component – mounted once in the root layout.
export const LampIcon: React.FC;

// Screen – resources list, full-screen modal.
// Loaded from bundled JSON.
export const LampScreen: React.FC;

// The locale function used at open-time:
export function resolveLampLocale(): string;
// returns e.g. 'en-AU' based on OS locale and country region.
// never sends an IP-based lookup.
```

Data

Resource lists are JSON files in assets/lamp-resources/, one per locale. Each file is signed (SHA-256 manifest, committed to the repo) so that a tampered bundle is detectable during release signing.

```
// en-AU.json shape
{
  "version": "2026-04-20",
  "locale": "en-AU",
  "reviewed_at": "2026-03-14",
  "resources": [
    {
      "category": "suicide_crisis",
      "name": "Lifeline Australia",
```

```

    "phone": "13 11 14",
    "sms": null,
    "url": "https://www.lifeline.org.au",
    "hours": "24/7"
  },
  ...
]
}

```

Dependencies

- expo-localization

Notes

The Lamp is a compliance surface, not a product surface.

Every other module in this app has a 'cut for time' option. The Lamp does not. If Phase 1 ships without working offline on a phone in aeroplane mode with the locale set to an unsupported region, the release is blocked. The fallback for unsupported locales is the en-default resource list with a clear 'your region is not fully supported yet' header — never an empty screen, never a 'please try again' message.

- No splash, no loader, no 'tap to view' state. The modal appears immediately; the data is already bundled.

Risks

- Editorial risk — the resource list must be reviewed quarterly (brief §5.9.2). The app contains a build-time assertion that fails CI if reviewed_at is older than 180 days at release time.

Acceptance criteria

- Lamplcon is rendered on every screen in the (moat), (rings), and (lamp) route groups; a component tree test enforces this.
- Opening the Lamp in aeroplane mode renders the full resource list with no network request (verified via a Charles Proxy run in CI).
- No call to any logger, analytics SDK, or server originates from the Lamplcon or LampScreen component trees (verified by import-scan static analysis).
- When the OS locale is one we do not ship, the en-default list is shown with the fallback banner.

4.11 infra.logging — the body-stripping logger

Responsibility

Provide a single logger interface for the app. Strip any value that looks like message content. Upload only aggregated metric-shape events to the ops store (Grafana Cloud). Refuse, by static type, to accept a 'body' field.

Public interface

```

type MetricEvent = {
  name: string;
  // 'onboarding.step.complete', etc.
}

```

```

  attrs: Record<string, string | number>; // small, shape-bounded
  // Note: no 'body', no 'content', no 'text' ever.
};

export function emit(event: MetricEvent): void;

// Pre-commit hook additionally scans calls to any console.* and rejects
// a commit that includes console.log outside the logging module.

```

Data

Events are batched in memory and flushed every 60 seconds. If the network is unreachable, events older than 24 hours are dropped (never persisted to disk).

Dependencies

- OpenTelemetry SDK (RN) with OTLP HTTP exporter

Notes

- A bespoke ESLint rule bans string-concatenation-into-emit of anything that is not a literal or a small enum. Every emit call site is reviewed in PRs for shape.

Risks

- Transitive telemetry — some third-party SDKs emit their own network traffic. The scripts/patch-transitive.ts script patches known offenders at build time; unknown ones are caught by Charles Proxy in CI.

Acceptance criteria

- The literal strings 'body', 'content', 'text' do not appear as a key in any emit() call site in the codebase (grep + AST check).
- Running the app through a filtering proxy in CI produces zero unknown hosts beyond the allowlist.

4.12 infra.notifications — silent push registration

Responsibility

Register for APNs (iOS) and FCM (Android) silent-push tokens. Send the token to the backend. Handle incoming silent pushes as wake-up signals only. Never show a notification with content. Never attach a body.

Public interface

```

export async function registerForSilentPush(): Promise<string>; // token
export async function refreshTokenIfChanged(): Promise<void>;
export function onWakeup(handler: () => Promise<void>): Unsubscribe;

```

Data

Token is stored in secure store under moatly.push-token. Backend receives the token via the auth endpoint.

Dependencies

- expo-notifications

Notes

- Silent push only. No alerts. No badges. No sounds. No content.
- On iOS, silent pushes are throttled — this is a known constraint from the brief (§14.1).

Risks

- User opts out of notifications entirely — fine, fall back to foreground polling when the app is open.

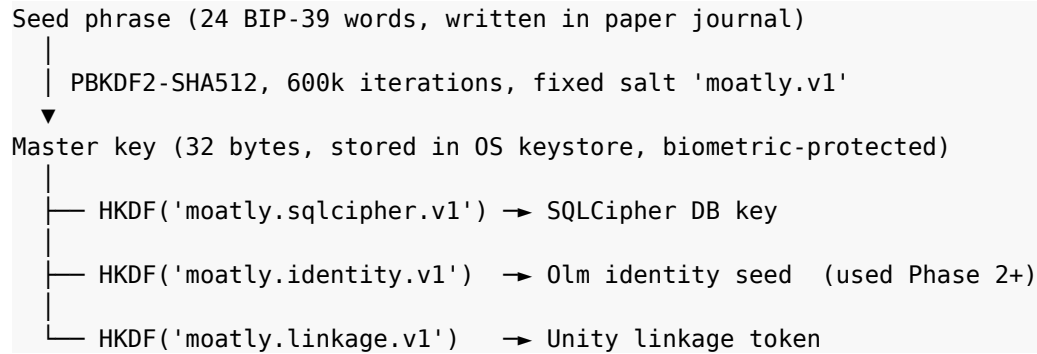
Acceptance criteria

- A Maestro test verifies no system notification banner ever appears as a result of any server-sent message.
- Push payloads contain at most { wakeup: true } — no other keys.

5. Cross-cutting concerns

5.1 Key lifecycle

The diagram below shows how a single seed phrase derives everything else on the device. Nothing persists outside the OS keystore and the encrypted SQLCipher file.



The seed phrase never touches the keystore, never touches the DB, never leaves the app's memory after onboarding. It lives exactly in two places: the user's paper journal, and the user's head.

5.2 State management

Three stores, plus React Query for server state (relevant from Phase 2).

Store	Purpose	Notes
useOnboardingStore	Current onboarding step; covenant hash; completion bitmap.	Hydrated from DB at app start; writes-through to DB.
useCompassStore	Current cycle's compass + lock status.	Hydrated lazily when user opens Moat home.
useAppSessionStore	App-lifecycle state: is-keystore-unlocked, last-background-at.	In-memory only; never persisted.

5.3 Navigation

- Root stack: an auth-state-dependent router — unonboarded, onboarded-no-moat, onboarded-with-moat.
- No deep linking into Moat screens. A notification tap lands at the app root, which then decides where to go based on state.
- The Lamp is a modal presented over any screen in the (moat) / (rings) route groups. It is not a route in the stack.

5.4 Offline behaviour

- The Lamp works fully offline — bundled resource list.
- Onboarding works fully offline up to the email-relay step. Placement (step 7) requires network in Phase 2; in Phase 1 it is a local no-op.
- Compass submission works offline — it is a local DB insert.

- Email relay requires network. Queue-and-retry is not implemented in Phase 1; the UI shows 'sent when online' and retries on next foreground.

5.5 Error handling

- Errors thrown inside Layer 1 (crypto) are opaque — we never surface a crypto error verbatim. Users see 'we couldn't unlock; use your seed phrase if this persists.'
- Errors in Layer 2 that are retryable (network) are retried with exponential backoff once; the second failure surfaces a banner. No silent retries beyond that.
- Errors in Layer 3 (domain) bubble to the UI as typed results, not exceptions.
- Errors that indicate data corruption are fatal — show a modal, require seed-phrase recovery.

5.6 Accessibility

- VoiceOver and TalkBack parity are non-negotiable. Tap-and-hold interactions are replaced with explicit accept buttons when a screen reader is active.
- Dynamic type support up to the OS maximum.
- No colour-only signalling. Every state has both a colour and a shape/text cue.
- The Lamp is the single most important accessibility surface — it must work with any accessibility setting enabled.

6. Data flows for the Phase 1 scenarios

Described as sequences. Each flow lists the modules involved and the ordered steps. Read these after the module specs; they are the glue.

6.1 First-run onboarding (happy path)

1. App launches. `useAppSessionStore.keystoreUnlocked = false`.
2. Root router: `crypto.keyring.loadMasterKey()` → null → route to `/threshold`.
3. User taps through threshold → `markStepComplete('threshold')`.
4. User reads declaration → scroll-to-end → `markStepComplete('declaration')`.
5. User tap-and-holds all three covenant clauses.
→ `compute SHA-256(covenant-text-v1)` → `save covenant_hash`
→ `markStepComplete('covenant')`.
6. Phase 1 skips `device_pairing` (creates placeholder virtual device).
7. `crypto.seed.generateSeedPhrase()` → 24 words.
User writes them in the journal.
App displays 3 randomised confirmation prompts.
On correct confirmation:
 `crypto.keyring.deriveAndStoreMasterKey(seed)`
 `infra.db.openDatabase(masterKey)`
 `markStepComplete('seed_phrase')`.
8. Two Questions screen → user submits `whatToFix + whyForWhom`.
`domain.compass.submitCompass(..., cycleNumber=0)`.
`markStepComplete('two_questions')`.
9. Placement screen: Phase 1 shows
 "your first Moat will form when Moats are turned on."
 `markStepComplete('placement')`.
10. Root router re-evaluates → routes to `(moat)/ placeholder home`.
`useAppSessionStore.keystoreUnlocked = true`.

6.2 Returning user opens the app

1. App launches. `useAppSessionStore.keystoreUnlocked = false`.
2. Root router: `crypto.keyring.loadMasterKey()` → `MasterKeyHandle`.
This triggers a biometric prompt on both platforms.
3. On success: `infra.db.openDatabase(masterKey)` → hydrates stores.
4. `useOnboardingStore.isOnboarded ? route to (moat) : route to /threshold`.
5. `useAppSessionStore.keystoreUnlocked = true`.
6. On background (OS event), clear the in-memory app session after 5 minutes of inactivity. Next foreground requires step 2 again.

6.3 User submits a ritual prescription via the Unity app

1. Unity app calls into our deep link with a signed envelope.
2. `moat-app` receives the envelope in a background handler, validates the signature against the Unity linkage token.
3. `domain.linkage.relayPrescription(envelope)`:
 - `POST /v1/linkage/relay_prescription`
 - On 202 Accepted: write `{ relayId, accountId, acceptedAt }` to `linkage_receipts`. `bodyPlain` and `bodyHtml` are discarded in the same call stack frame.
4. On failure: show a banner "prescription will retry on next open."

- A single retry is scheduled for app foreground.
- No persistent queue – if the app is uninstalled before the retry fires, the prescription is lost. Accepted trade-off.

6.4 User taps the Lamp

1. LampIcon.onPress → navigation.openModal('lamp').
2. LampScreen mounts.
 resolveLampLocale() → e.g. 'en-AU'.
3. Load assets/lamp-resources/en-AU.json synchronously (require()).
4. Render list.
5. User dismisses → modal closes.

At NO point in steps 1–5:

- does any emit() call fire,
- does any network request originate,
- does any write happen to the DB or secure store,
- does any analytic, trace, or log line enter any sink.

7. Test strategy

7.1 The pyramid

Level	Tool	Scope	Target count at end of Phase 1
Unit	Jest	Pure functions in Layer 3; schema constraints.	~400 tests
Property	fast-check	Seed, keyring, rings computation.	~15 properties, 10k iters each
Integration	Jest + sqlite in-memory	DAOs against real SQLCIPHER.	~60 tests
E2E	Maestro	Onboarding happy path, Lamp-offline, key lifecycle.	~12 flows
Privacy	Charles Proxy in CI	Allow-list of hosts; zero egress from Lamp.	1 gate test per release
Static	ESLint + bespoke rules	No telemetry leaks, no cross-layer imports.	Continuous

7.2 Three tests that must exist before Phase 1 exits

- **The DB-opacity test:** take a copy of the SQLCIPHER file from a real install, attempt to open with vanilla sqlite, expect failure.
- **The Lamp-offline test:** boot the app with the network disabled, tap the Lamp, observe that resources render and zero network attempts are made.
- **The no-body-leak test:** generate 100 known-string prescription envelopes, relay them, then grep every logger sink, the DB file, the disk cache, and the metric batch buffer for any occurrence of any known string. Expect zero hits.

8. Implementation plan — Phase 1

The Build Brief's Phase 1 budgets 10–12 weeks with two mobile engineers. The plan below walks that budget. Weeks are 5-day weeks at 6 hours of focused engineering per engineer per day (the usual allowance for meetings, review, interruptions).

8.1 Weekly plan

Week	Mobile engineer A (foundation)	Mobile engineer B (UI and ritual)
1	Repo scaffold, Expo prebuild, CI, proxy gate.	Design-tokens package; tap-and-hold component prototype.
2	crypto.seed + property tests.	Threshold, Declaration screens; Lamp icon shell.
3	crypto.keyring + keystore tests.	Covenant screen with 3 clauses, with a11y fallback.
4	infra.secure-store + ESLint rules.	Lamp resource bundle & LampScreen (en-GB, en-AU, en-US).
5	infra.db + migrations + DB-opacity test.	Lamp locale fallback + offline E2E test.
6	domain.onboarding state machine + state tests.	Seed-phrase UI (generation + 3-word prompts).
7	domain.compass + 24h-lock tests.	Two Questions UI + character-limit UX.
8	domain.rings + ring-computation property tests.	Rings preview screen (Phase 1 sparse state).
9	domain.linkage + email-relay client.	Placement placeholder screen + account-delete flow.
10	infra.logging + no-body-leak test.	Accessibility pass across all screens.
11	infra.notifications + silent push registration.	Polish pass: animations, haptics, dark mode.
12	Dogfood bug-burn & release candidate.	Dogfood bug-burn & release candidate.

8.2 Phase 1 exit gate (verbatim from the brief)

- 10 internal dogfooders complete 2 full 7-day cycles.
- Zero P1 bugs open.
- Crash-free rate $\geq 99.5\%$ on both iOS and Android.

Plus the three named tests in §7.2 of this document, which are mobile-specific and will be proposed as additions to the brief's gate.

9. Open questions — decisions needed this week

Listed with owner, trigger (when the answer is needed), and default-if-no-answer.

Question	Owner	Needed by	Default if no answer
React Native or Flutter? (§1.1)	Lead	Before repo scaffold (Week 1)	Use RN.
Expo prebuild or bare RN? (§1.2)	Lead	Same	Use Expo prebuild.
Confirm covenant text v1 wording.	Founder	Before Week 3	Use the three-clause wording from brief §5.1.1 verbatim.
Which countries does the Lamp ship with at Phase 1?	Product + legal	Before Week 4	GB, AU, US, IE, DE, FR. Others fall back to en-default.
Who curates the Lamp resource list?	Product	Before Week 4	Product + clinical advisor identified by founder.
Unity app linkage contract — is the token format and envelope exactly as in brief §5.3.3?	Unity lead + Moatly lead	Before Week 9	Implement to brief. If Unity drifts, fix in a cross-team PR.
What is 'cycle 0' during Phase 1, given no Moats exist?	Lead	Before Week 7	Cycle 0 = the user's first 7 days post-onboarding. Reset to match Moat cycle on Phase 2 placement.
Do we ship Phase 1 without an email-relay UI surface at all, or with a 'recent prescriptions' list visible?	Product	Before Week 9	Ship with a bare 'last 7 delivered' list — no bodies.

10. What comes next

After Phase 1 exits, two follow-up technical breakdowns will be produced. Each will match the depth of this one.

10.1 Phase 2 breakdown (Moat service client, Megolm via vodozemacs)

- The vodozemacs FFI boundary — what crosses JS ↔ Rust, what doesn't.
- Megolm group session management on the client side.
- Cycle-boundary session rotation.
- The Moat home screen, whisper/witness/major-statement composition.
- WebSocket client, reconnection, message ordering guarantees.
- Complaint flow UI and encrypted envelope composition.
- Soft-auth (email + seed phrase) as a time-bounded stand-in.

10.2 Phase 3 breakdown (Device integration)

- BLE pairing flow on top of react-native-ble-plx.
- The combination capture UI (capacitive ring for Premium, press-pattern for Simple, on-screen for Virtual).
- Composite identity code composition and server exchange.
- Glow driver — the 5 defined events, the silent rest state.
- Signed firmware OTA initiated from the app.
- Decommissioning the soft-auth path from Phase 2.

10.3 Ongoing — the online portal

A separate breakdown, in progress, covers the marketing / waitlist / journal-partner / transparency-reports portal. That portal is the public face of Moatly; it does not share code with the Moat app and is designed to ship earlier (roughly Phase 1 parallel) so the waitlist and the first cohort comms can go out before the Moats exist.

— End of Technical Breakdown —

Moatly · Moat App technical breakdown · April 2026 · v0.1